High Performance Computing Lab - MPI Assignments

Course: 23CS302PC303

Name : N.varshith

HT No : 2303A51441

Batch No : 21

Assignment 1: Basic Point-to-Point Communication using MPI

Scenario: A distributed logging system needs to transfer status messages from a master process to a worker process.

Python Code (assignment1.py)

```
from mpi4py import MPI

def main():
    # Initialize the communicator
    comm = MPI.COMM_WORLD
    rank = comm.Get_rank()
    size = comm.Get_size()

    if size < 2:
        if rank == 0:
            print("Error: This program requires at least 2 processes.")
        return

    if rank == 0:
        # Master process
        message = "System status: ALL SYSTEMS GO!"
        # Send message to rank 1
        comm.send(message, dest=1, tag=11)
        print(f"[Rank 0] Sent message: '{message}' to Rank 1")

    elif rank == 1:
        # Worker process
        # Receive message from rank 0
        received_msg = comm.recv(source=0, tag=11)
```

```
        print(f"[Rank 1] Received message: '{received_msg}' from Rank 0")

if __name__ == "__main__":
    main()
```

Execution Command

mpiexec -n 2 python assignment1.py

Output

[Rank 0] Sent message: 'System status: ALL SYSTEMS GO!' to Rank 1
[Rank 1] Received message: 'System status: ALL SYSTEMS GO!' from Rank 0

Assignment 2: Integer Data Exchange between Processes

Scenario: A parallel sensor network distributes integer readings from a control node to compute nodes.

Python Code (assignment2.py)

```
from mpi4py import MPI

def main():
    comm = MPI.COMM_WORLD
    rank = comm.Get_rank()
    size = comm.Get_size()

    if size < 4:
        if rank == 0:
            print("Error: Please run with at least 4 processes.")
        return

    if rank == 0:
        # Send different integers to ranks 1, 2, and 3
        sensor_data = {1: 1024, 2: 2048, 3: 4096}

        for dest_rank, val in sensor_data.items():
```

```
        # Using the destination rank as the tag for differentiation
        comm.send(val, dest=dest_rank, tag=dest_rank)
        print(f"[Rank 0] Sent value {val} to Rank {dest_rank} with tag {dest_rank}")

    elif rank in [1, 2, 3]:
        # Receive data matching this process's rank as the tag
        received_val = comm.recv(source=0, tag=rank)
        print(f"[Rank {rank}] Received sensor value {received_val} from Rank 0")

if __name__ == "__main__":
    main()
```

Execution Command

```
mpiexec -n 4 python assignment2.py
```

Output

```
[Rank 0] Sent value 1024 to Rank 1 with tag 1
[Rank 0] Sent value 2048 to Rank 2 with tag 2
[Rank 0] Sent value 4096 to Rank 3 with tag 3
[Rank 1] Received sensor value 1024 from Rank 0
[Rank 2] Received sensor value 2048 from Rank 0
[Rank 3] Received sensor value 4096 from Rank 0
```

Assignment 3: One-to-One Communication Pattern

Scenario: Each process exchanges boundary data with its neighboring process (Even to Odd).

Python Code (assignment3.py)

```
from mpi4py import MPI

def main():
    comm = MPI.COMM_WORLD
    rank = comm.Get_rank()
    size = comm.Get_size()

    if size % 2 != 0:
        if rank == 0:
```

```
        print("Error: Please run with an even number of processes.")
    return

    if rank % 2 == 0:
        # Even ranks send data to the next odd rank (rank + 1)
        dest = rank + 1
        boundary_data = f"Boundary Matrix from Node {rank}"
        comm.send(boundary_data, dest=dest, tag=99)
        print(f"[Rank {rank} - EVEN] Sent data to Rank {dest}")

    else:
        # Odd ranks receive data from the previous even rank (rank - 1)
        source = rank - 1
        received_data = comm.recv(source=source, tag=99)
        print(f"[Rank {rank} - ODD] Received: '{received_data}' from Rank {source}")

if __name__ == "__main__":
    main()
```

Execution Command

mpiexec -n 4 python assignment3.py

Output

[Rank 0 - EVEN] Sent data to Rank 1
[Rank 2 - EVEN] Sent data to Rank 3
[Rank 1 - ODD] Received: 'Boundary Matrix from Node 0' from Rank 0
[Rank 3 - ODD] Received: 'Boundary Matrix from Node 2' from Rank 2

Assignment 4: Deadlock Analysis in MPI Send/Receive

Scenario: A parallel application experiences a hang due to improper ordering of communication calls.

Deadlock Explanation

If two processes (Rank 0 and Rank 1) both execute comm.send() sequentially before executing comm.recv(), and if the messages are large enough to bypass standard system buffers, both processes will block indefinitely. They are both waiting for the other to start receiving, creating

a **deadlock**.

To fix this, we reorder the calls: one process must initiate the send, while the other initiates the receive.

Python Code (assignment4.py - Corrected)

```
from mpi4py import MPI

def main():
    comm = MPI.COMM_WORLD
    rank = comm.Get_rank()

    if comm.Get_size() < 2:
        return

    # To avoid deadlock, we ensure complementary communication ordering.
    # If Rank 0 sends then receives, Rank 1 MUST receive then send.

    if rank == 0:
        # 1. Rank 0 Sends first
        comm.send("Ping from 0", dest=1, tag=1)
        print("[Rank 0] Sent message to Rank 1")

        # 2. Rank 0 Receives second
        msg = comm.recv(source=1, tag=2)
        print(f"[Rank 0] Received: {msg}")

    elif rank == 1:
        # 1. Rank 1 Receives first
        msg = comm.recv(source=0, tag=1)
        print(f"[Rank 1] Received: {msg}")

        # 2. Rank 1 Sends second
        comm.send("Pong from 1", dest=0, tag=2)
        print("[Rank 1] Sent message to Rank 0")

if __name__ == "__main__":
    main()
```

Execution Command

mpiexec -n 2 python assignment4.py

Output

[Rank 0] Sent message to Rank 1
[Rank 1] Received: Ping from 0
[Rank 1] Sent message to Rank 0
[Rank 0] Received: Pong from 1

Assignment 5: Master-Worker Communication

Scenario: Master distributes tasks to workers and collects results.

Python Code (assignment5.py)

```
from mpi4py import MPI
import time

def main():
    comm = MPI.COMM_WORLD
    rank = comm.Get_rank()
    size = comm.Get_size()

    if size < 2:
        if rank == 0:
            print("Error: Master-Worker model requires at least 2 processes.")
        return

    if rank == 0:
        # MASTER PROCESS
        print(f"[Master] Distributing tasks to {size - 1} workers...")

        # 1. Send tasks to all workers
        for worker_id in range(1, size):
            task_payload = worker_id * 5  # Example data
            comm.send(task_payload, dest=worker_id, tag=100)
            print(f"[Master] Sent task data ({task_payload}) to Worker {worker_id}")

        # 2. Collect results from all workers
        print("[Master] Waiting for results...")
```

```
        for worker_id in range(1, size):
            result = comm.recv(source=worker_id, tag=200)
            print(f"[Master] Collected result ({result}) from Worker {worker_id}")

    else:
        # WORKER PROCESS
        # 1. Receive task from master
        task_data = comm.recv(source=0, tag=100)
        print(f"[Worker {rank}] Received task data ({task_data}). Computing...")

        # Simulate computation (e.g., squaring the number)
        time.sleep(0.5)
        computation_result = task_data ** 2

        # 2. Send result back to master
        comm.send(computation_result, dest=0, tag=200)
        print(f"[Worker {rank}] Sent result ({computation_result}) back to Master")

if __name__ == "__main__":
    main()
```

Execution Command

mpiexec -n 4 python assignment5.py

Output

```
[Master] Distributing tasks to 3 workers...
[Master] Sent task data (5) to Worker 1
[Master] Sent task data (10) to Worker 2
[Master] Sent task data (15) to Worker 3
[Master] Waiting for results...
[Worker 1] Received task data (5). Computing...
[Worker 2] Received task data (10). Computing...
[Worker 3] Received task data (15). Computing...
[Worker 1] Sent result (25) back to Master
[Worker 2] Sent result (100) back to Master
[Worker 3] Sent result (225) back to Master
[Master] Collected result (25) from Worker 1
[Master] Collected result (100) from Worker 2
```

[Master] Collected result (225) from Worker 3